

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4, BL5)

Debugging & Optimisation Task: 10%

Total Marks: 50

Annexure A

Code of Conduct:

I Dhanush Kumar S(192421096) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Dhanush Kumar S

Question 1: Debugging Hash Function Implementation (SHA Family)

Scenario: A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

Errors identified in padding, block processing, bitwise ops & endianness

- Endianness bug: the 512-bit message chunks were unpacked into 32-bit words using little-endian byte order ('<16I'), but the SHA-256 specification requires big-endian ('>16I') word extraction.
- Padding bug: the 64-bit original-message-length suffix was packed little-endian ('<Q') instead of the required big-endian ('>Q').
- Bitwise-operation bug: the right-rotate helper did not mask its result back to 32 bits (no '& 0xFFFFFFFF'), so intermediate values grew into arbitrarily large Python integers instead of wrapping at 32 bits as the algorithm requires.

Buggy right-rotate and word extraction:

```
def _rrot_buggy(x, n):
    return (x >> n) | (x << (32 - n))          # BUG: no 32-bit mask -> unbounded growth

msg += struct.pack('<Q', len(message) * 8)      # BUG: length must be big-endian ('>Q') w =
list(struct.unpack('<16I', chunk)) + [0]*48      # BUG: words must be big-endian ('>16I')
```

Fixed versions:

```
def _rrot(x, n):
    return ((x >> n) | (x << (32 - n))) & 0xFFFFFFFF # FIX: mask to 32 bits every time

msg += struct.pack('>Q', len(message) * 8)          # FIX: big-endian length w =
list(struct.unpack('>16I', chunk)) + [0]*48         # FIX: big-endian words
```

Verification against Python's hashlib (ground truth)

input	buggy_matches?	fixed_matches? (empty)
False	True	
hello	False	True
The quick brown fox jumps over the lazy ...	False	True
aa...	False	True

The corrected implementation matches hashlib.sha256 exactly on every test input; the buggy version never matches, confirming all three fixes were necessary and sufficient.

Optimization for large files (≥ 50 MB)

The original implementation read the entire file into memory before hashing, which does not scale to large files. It was rewritten to stream the file in fixed-size chunks so memory use stays constant regardless of file size, and to use hashlib's C-optimized SHA-256 for the actual digest computation (a from-scratch pure-Python SHA-256 is orders of magnitude slower than the C implementation and is unsuitable for real multi-megabyte files):

```
def hash_large_file(path, chunk_size=1024*1024):
    h = hashlib.sha256()
    with open(path, 'rb') as f:
        for block in iter(lambda: f.read(chunk_size), b''):
            h.update(block)          # constant memory; C-accelerated per chunk
    return h.hexdigest()
```

```
60MB file - whole-file load: 0.081s | chunked (1MB buffer): 0.058s | match: True
```

Streaming in 1 MB chunks was both faster and used a small, constant amount of memory instead of holding the entire 60 MB file in RAM at once — the gap widens further as file size grows.

Secure hash functions vs. basic checksums — evaluation

A basic checksum (e.g., CRC32) is designed to catch accidental transmission errors, not deliberate tampering: it is linear, so an attacker can compute how to modify the data and the checksum together to produce any desired checksum value, making forgery trivial. A cryptographic hash such as SHA-256 provides preimage resistance (infeasible to find an input producing a given hash), second-preimage resistance, and collision resistance, and exhibits the avalanche effect (a one-bit input change flips roughly half the output bits unpredictably). This means an attacker cannot feasibly modify data while preserving its SHA-256 digest, making cryptographic hashes suitable for security-relevant integrity checks, while checksums are only suitable for detecting accidental corruption.

Question 2: Optimization of SSL/TLS Handshake Simulation

Scenario: A Python-based simulation of the SSL/TLS handshake shows delays and occasional handshake failures.

Debugging certificate validation and key exchange

- Certificate validation bug: the validator returned True unconditionally, without checking the certificate's digital signature or its expiry ('not_after') timestamp — so a forged or expired certificate would be accepted, a serious security bug.
- Key exchange bug: the client generated its own Diffie-Hellman group parameters (p, g) independently instead of using the group published in the server's certificate. Whenever the client's guessed parameters didn't match the server's, the two sides computed different shared secrets and the handshake failed — explaining the 'occasional' failures.

```
def validate_cert_buggy(cert):
    return True # BUG: no signature check, no expiry check

def client_handshake_buggy(server_cert):
    p, g = 23, 5 # BUG: ignores server_cert['p'] / ['g'] -- picks its own group
```

Fixes applied — signature and expiry are checked, and the client adopts the server's own group parameters:

```
def validate_cert_fixed(cert):
    fields =
    f'{cert["subject"]}{cert["p"]}{cert["g"]}{cert["server_pub"]}{cert["not_after"]}'.encode()
    sig_ok =
    hmac.compare_digest(ca_sign(fields), cert["sig"])
    not_expired = time.time() < cert["not_after"]
    return sig_ok and not_expired

def client_handshake_fixed(server_cert):
    if not validate_cert_fixed(server_cert):
        raise ValueError("invalid or expired certificate")
    p, g = server_cert["p"], server_cert["g"] # FIX: use the server's actual group
```

```
Buggy handshake succeeded: 9/200 times
Fixed handshake succeeded: 200/200 times
Forged cert accepted by buggy validator? True
Forged cert accepted by fixed validator? False
Expired cert accepted by fixed validator? False
```

Optimizing the handshake — session resumption

A session cache was added, keyed by session ID, storing the derived master secret from the first (full) handshake. On a repeat connection, the client and server look up the cached secret instead of repeating the full certificate exchange and Diffie-Hellman computation — an abbreviated handshake:

```
def resumable_handshake(session_id, server_p, server_g, server_priv):
    if session_id in SESSION_CACHE:
        return SESSION_CACHE[session_id]      # OPTIMIZATION: reuse master secret    ok =
    full_handshake(server_p, server_g, server_priv)
    SESSION_CACHE[session_id] = ok
    return ok
```

```
First connection (full handshake): 0.0502s
Repeat connection (session resumed): 0.0011s (~44x faster)
```

Performance vs. security trade-offs in TLS optimization

Session resumption dramatically reduces latency (roughly 44× faster in this simulation) by skipping the expensive asymmetric key exchange on repeat connections. The trade-off is that reusing the same master secret for an extended period widens the exposure window if that secret is ever compromised, and can weaken forward secrecy (past sessions could be exposed if a long-lived resumed secret is broken). In practice this is mitigated by giving session tickets a short expiry, rotating session-ticket encryption keys frequently, and preferring ephemeral Diffie-Hellman (DHE/ECDHE) for the initial full handshake so each full handshake still generates an independent secret even though resumed sessions reuse it briefly.

Question 3: Debugging and Optimizing Digital Signature Algorithm (DSA)

Scenario: A Python implementation of DSA generates invalid signatures during verification.

Issues in parameter selection and random number generation

- Random number bug: the per-signature secret k was drawn using `random.randint(0, q-1)`, which (a) uses $k = 0$, Python's non-cryptographic PRNG and (b) can return $k = 0$, for which no modular inverse exists.
- Signature generation bug: s was computed modulo p instead of modulo q as DSA requires, so s frequently fell outside the valid range $[1, q)$ expected by the verifier, causing verification to fail.

```
k = random.randint(0, q - 1)          # BUG: allows k = 0; not a CSPRNG
s = (kinv * (H(m) + x * r)) % p        # BUG: should be mod q, not mod p
```

Fixes — correct modulus, valid k range, and a cryptographically secure random source:

```
k = secrets.randbelow(q - 1) + 1      # FIX: k in [1, q-1], CSPRNG (secrets module)
s = (kinv * (H(m) + x * r)) % q        # FIX: correct modulus q
```

Verification results over repeated trials

```
Buggy DSA: 0/200 signatures verified successfully Fixed
DSA: 200/200 signatures verified successfully
```

The wrong modulus in the buggy version caused every single signature to fail verification; after correcting the modulus and the random- k generation, all 200 trial signatures verified successfully.

Optimizing modular exponentiation

Signature generation/verification needs repeated modular exponentiation ($g^k \bmod p$, $g^{u1} \bmod p$, etc.). A naive implementation multiplying in a loop runs in $O(\text{exponent})$ time; replacing it with Python's built-in `pow(base, exp, mod)`, which uses square-and-multiply, reduces this to $O(\log \text{exponent})$:

```
def modexp_naive(base, exp, mod):
    result = 1
    for _ in range(exp):          # O(exp) -- infeasible for real (large) exponents
        result = (result * base) % mod
    return result

r2 = pow(base, exp, mod)         # O(log exp) -- square-and-multiply
```

```
modexp_naive: 2 in 0.000255s | pow() fast-modexp: 2 in 0.000002s match=True
```

Even with a deliberately small exponent for demo purposes, the built-in fast exponentiation was roughly two orders of magnitude faster; with the large exponents real DSA uses (hundreds of bits), the naive loop would be completely infeasible while `pow()` remains fast.

Impact of randomness quality on signature security

DSA's security depends critically on k being unpredictable and never reused. If k is generated by a weak or predictable PRNG, or if the same k value is ever reused across two different messages, an attacker who obtains both signatures can algebraically solve for k and then recover the private key x directly from the signature equations. This is not a theoretical concern — real-world private key recoveries (e.g., the Sony PlayStation 3 ECDSA incident) resulted from exactly this kind of nonce reuse. This is why the fix uses Python's `secrets` module (a cryptographically secure PRNG) rather than the `random` module, and regenerates k whenever a degenerate value ($r = 0$ or $s = 0$) is produced.

Question 4: Debugging Key Management System

Scenario: A Python-based key management system fails to securely store and retrieve cryptographic keys.

Flaws identified in key storage

- Keys were written to disk as plain hex strings inside a human-readable JSON file — anyone with file-system access (or a backup copy) could read every key directly.
- `get_key()` returned any requested key to any caller with no authentication or authorization check at all.
- There was no concept of key expiry or rotation — once generated, a key was used indefinitely.

```
self.keys[key_id] = {"key": os.urandom(32).hex()} # BUG: stored as plain hex, no encryption
def get_key(self, key_id, requester=None):
    return self.keys[key_id]["key"] # BUG: no permission check at all
```

Fixes — encrypted storage, access control, and lifecycle management

```
def generate_key(self, key_id):
    raw_key = os.urandom(32)
    self.keys[key_id] = {
        "active": True, "created": time.time(),
        "enc_key": self.fernet.encrypt(raw_key).decode(), # FIX: encrypted before storage
    }
```

```

def get_key(self, key_id, requester_role="admin"):
    if requester_role not in ("admin", "service"):          # FIX: access control      raise
        PermissionError(f"role '{requester_role}' is not authorized")    raw_key =
    self.fernet.decrypt(self.keys[key_id]["enc_key"].encode())
    return raw_key.hex()

def rotate_key(self, key_id):                                # FIX: lifecycle management self.keys[key_id]
    ["active"] = False    # retire old key, kept only for decrypting old data new_id = f"{key_id}
    _v{int(time.time())}"
    self.generate_key(new_id)                                # generate a fresh active key
    return new_id

```

Demonstration output

```

BUGGY store on disk (plaintext, readable by anyone with file access):
{"db_key": {"key": "5e60802c515163d07bcb5b54440b4a9327ffca64b5a4502bf9accb54bfd7b663"}} BUGGY
get_key with NO credentials at all:
5e60802c515163d07bcb5b54440b4a9327ffca64b5a4502bf9accb54bfd7b663

FIXED store on disk (ciphertext only):
{"db_key": {"active": true, "created": 1785488858.02, "enc_key": "gAAAAABqbGXaw...
(encrypted)"}}
FIXED get_key with unauthorized role correctly rejected: role 'guest' is not authorized to fetch keys
FIXED get_key with authorized role:
f53aaf31b2f6e132b5d1734262fd0aabdb3ec0fcd39baf418615dab3fb2286a
FIXED rotate_key: old 'db_key' retired, new active key created as 'db_key_v1785488858'

```

The buggy store exposes the raw key in cleartext to anyone who can read the file and imposes no access restriction; the fixed store only ever persists ciphertext (using Fernet envelope encryption under a master key kept outside the store), enforces a role check before returning a decrypted key, and supports rotating a key while retaining the retired version only for decrypting previously-encrypted data.

Impact of poor key management on the overall system

Cryptographic algorithms such as AES or RSA can be mathematically unbreakable in practice, but the entire system's security ultimately reduces to the secrecy of its keys. If keys are stored in plaintext, leaked through weak access control, or never rotated, an attacker does not need to break the cipher at all — they simply steal the key and can decrypt all past and future data protected by it, forge signatures, or impersonate legitimate parties. Poor key management is consistently one of the most common root causes of real-world cryptographic breaches, precisely because it bypasses the strength of the underlying algorithm entirely; the key management system is only as strong as its weakest storage, access-control, or rotation practice.

Question 5: Optimization of Secure File Transfer Protocol (SFTP-like System)

Scenario: A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption.

Debugging encryption/decryption synchronization and packet handling

- Nonce-reuse bug: every chunk was encrypted with AES-GCM using the same fixed nonce. Reusing a nonce with the same key is a critical cryptographic error that breaks the security guarantees of GCM (and would leak plaintext relationships across chunks in a real deployment).
- No integrity handling: when a chunk failed authentication (e.g., because it was corrupted or tampered with in transit), the receiver silently replaced it with zero bytes instead of raising an alert — corruption was never signalled to the application.
- Packet-ordering bug: chunks were reassembled in whatever order they arrived over the (simulated) unreliable network, rather than by an explicit sequence number, so out-of-order delivery silently corrupted the reconstructed file.

```
FIXED_NONCE = b"\x00" * 12                # BUG: never changes between chunks ct
= aes.encrypt(FIXED_NONCE, chunk, None)    # BUG: nonce reused every call ...
for ct in packets:                         # BUG: trusts arrival order blindly try:
    out += aes.decrypt(FIXED_NONCE, ct, None)
except Exception:
    out += b"\x00" * len(ct)               # BUG: silently pads on failure
```

Fixes — unique per-chunk nonce, propagated authentication failures, and sequence-based reassembly:

```
nonce = seq.to_bytes(12, "big")            # FIX: unique nonce per chunk (seq-derived) ct =
aes.encrypt(nonce, chunk, None)
packets.append((seq, ct))                  # FIX: sequence number travels with packet ...
packets_sorted = sorted(packets, key=lambda p: p[0]) # FIX: reorder by sequence number for seq, ct in
packets_sorted:
    out += aes.decrypt(seq.to_bytes(12, "big"), ct, None) # raises InvalidTag if tampered
```

Test results: corruption detection and out-of-order delivery

```
BUGGY : corrupted chunk silently replaced with zero bytes, no error raised FIXED :
corrupted chunk detected -> rejected: InvalidTag

BUGGY : file correct after reordering? False
FIXED : file correct after reordering? True
```

Optimizing throughput for large transfers (≥ 100 MB)

Throughput was improved by using 1 MB chunks (rather than very small packets, which add per-packet overhead), performing encryption with hardware-accelerated AES-GCM (via the cryptography library, which uses AES-NI when available), and avoiding any full-file buffering by streaming chunk-by-chunk:

```
100MB transfer (fixed, optimized): encrypt 0.50s, decrypt 1.00s, throughput
~66.5 MB/s, integrity OK: True
```

Comparison with insecure file transfer, and justification

Aspect	Plain FTP (insecure)	Optimized secure system
Confidentiality	None — data sent in cleartext	AES-GCM encrypted per chunk
Integrity	None — corruption/tampering undetected	GCM auth tag verified per chunk
Throughput (100MB)	Fast, but insecure	~66.5 MB/s, well under 2s overhead

Plain (insecure) file transfer such as classic FTP sends both credentials and file data in cleartext, performs no integrity verification, and is trivially subject to eavesdropping, tampering, and credential theft on any network path the traffic crosses. The optimized secure system adds confidentiality (AES-GCM encryption), integrity and authenticity (the GCM authentication tag, verified per chunk), and correct reassembly under real-world network conditions (sequence numbers), at a throughput cost of well under a second of overhead for a 100 MB file. Given modern hardware-accelerated AES, this security is essentially “free” compared to the catastrophic risk of transmitting sensitive files with no encryption or integrity checking at all — which is why virtually all modern file-transfer tools (SFTP, HTTPS-based transfer, rsync-over-SSH) default to authenticated encryption rather than plaintext transfer.